

Лабораторная работа: Статическая оптимизация и численный поиск экстремалей

Информация

ФИО_1, ИСУ_1:

ФИО_2, ИСУ_2:

ФИО_3, ИСУ_3:

```
In [109... import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from matplotlib import cm
```

```
In [ ]: variant_number = -1 # Измените на свой номер варианта
```

```
In [111... # Определение параметров варианта

def get_variant_params(v):
    import numpy as np
    np.random.seed(v)
    params = {
        # Для 1 и 3 части
        'a': np.random.randint(1, 5),
        'b': np.random.randint(1, 5),
        'c': np.random.randint(-10, 10),
        'd': np.random.randint(-10, 10),
        'e': 10,
        'constr_a': np.random.randint(2, 6),
        'R2': np.random.randint(4, 10),
        'p': np.random.randint(1, 4),
        'q': np.random.randint(1, 4),
        # Для 4 части
        'FREQ': np.random.uniform(0.3, 0.8),
        'A_weight': np.random.uniform(0.5, 1.5),
        'B_weight': np.random.uniform(1.0, 3.0),
        'x_start': -4.0, 'y_start': -4.0,
        'x_end': 4.0, 'y_end': 4.0,
        'N_nodes': 30
    }
    return params

params = get_variant_params(variant_number)
```

```
In [112... print("Ваши параметры:", params)
```

```
Ваши параметры: {'a': 3, 'b': 1, 'c': 0, 'd': 6, 'e': 10, 'constr_a': 4, 'R
2': 6, 'p': 1, 'q': 2, 'FREQ': 0.7863005569524466, 'A_weight': 0.95344924741
73122, 'B_weight': 2.2180849255225557, 'x_start': -4.0, 'y_start': -4.0, 'x_
end': 4.0, 'y_end': 4.0, 'N_nodes': 30}
```

Часть 1. Поиск глобального минимума на основе необходимого и достаточного условий

1.1. Теоретическая справка

Рассмотрим целевую функцию двух переменных: $J(x, y) = a x^2 + b y^2 + c x + d y + e$.

Необходимое условие экстремума: Точка (x^*, y^*) является стационарной, если градиент целевой функции обращается в нуль: $\nabla J(x^*, y^*) = \begin{pmatrix} \frac{\partial J}{\partial x} \\ \frac{\partial J}{\partial y} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$.

Достаточное условие минимума: Стационарная точка (x^*, y^*) является точкой локального минимума, если матрица Гессе $H = \begin{pmatrix} \frac{\partial^2 J}{\partial x^2} & \frac{\partial^2 J}{\partial x \partial y} \\ \frac{\partial^2 J}{\partial y \partial x} & \frac{\partial^2 J}{\partial y^2} \end{pmatrix}$ является **положительно определённой** в данной точке.

1.2. Безусловная оптимизация

Аналитическая часть. Рассмотрим $J(x, y) = a x^2 + b y^2 + c x + d y + e$.

! Место для аналитического решения (без ограничений) !

Запишите ваши вычисления:

$$\frac{\partial J}{\partial x} = \dots$$

$$\frac{\partial J}{\partial y} = \dots$$

$$(x^*, y^*) = \dots$$

$$H = \dots$$

$$J(x^*, y^*) = \dots$$

Численная проверка. Реализуйте функцию $J(x, y)$ и найдите её минимум, решив систему $\nabla J = 0$. Постройте контурный график и 3D-поверхность.

```
In [113... def J(x, y):
    # ===== ВАШ КОД ЗДЕСЬ =====
    return ...

def grad_J(x, y):
    # ===== ВАШ КОД ЗДЕСЬ =====
    dJdx = ...
    dJdy = ...
    return np.array([dJdx, dJdy])

def hessian_J(x, y):
    # ===== ВАШ КОД ЗДЕСЬ =====
    H = ...
    return H
```

```
In [114... # --- Нахождение стационарной точки ---
# Решите систему grad_J = 0 аналитически и подставьте значения:
x_star = ... # ваше значение
y_star = ... # ваше значение

# --- Проверка положительной определённости ---
H = ...
```

```
In [115... # --- Визуализация ---
...

```

1.3. Задание: Оптимизация с ограничением-равенством

Добавим ограничение в виде равенства: $c(x, y) = x + y - 4 = 0$.

Метод множителей Лагранжа. Составим функцию Лагранжа: $\mathcal{L}(x, y, \lambda) = J(x, y) + \lambda \cdot c(x, y)$.
 Необходимые условия: $\frac{\partial \mathcal{L}}{\partial x} = 0$, $\frac{\partial \mathcal{L}}{\partial y} = 0$, $\frac{\partial \mathcal{L}}{\partial \lambda} = 0$.

! Место для аналитического решения (ограничение-равенство)
 !

Запишите систему уравнений и решение:

$$\frac{\partial \mathcal{L}}{\partial x} = 0$$

$$\frac{\partial \mathcal{L}}{\partial y} = 0$$

$$c(x, y) = 0$$

$$(x^*, y^*, \lambda^*) = ?$$

$$J(x^*, y^*) = ?$$

```
In [116... # --- Численная проверка (ограничение-равенство) ---
...

```

```
In [117... # --- Визуализация с ограничением-равенством ---
...

```

! Вопрос !

Сравните значения J^* в безусловном и условном минимумах.
Почему J^* при наличии ограничения не может быть меньше? Дайте геометрическую интерпретацию.

Ваш ответ:

1.4. Оптимизация с ограничением-неравенством

Теперь добавим ограничение в виде неравенства: $c(x, y) = x^2 + y^2 - 4 \leq 0$.
Допустимая область — круг радиуса 2 с центром в начале координат.

Условия Каруша — Куна — Таккера (ККТ): $\nabla J + \mu \nabla c = 0$, $\mu \geq 0$, $\mu \cdot c(x, y) = 0$.
Здесь μ — множитель ККТ, а $\mu \cdot c = 0$ — условие дополняющей нежёсткости: либо ограничение неактивно ($c < 0$, тогда $\mu = 0$), либо ограничение активно ($c = 0$, тогда $\mu \geq 0$).

Задание:

1. Проверьте, лежит ли безусловный минимум внутри допустимой области.
2. Запишите систему ККТ при активном ограничении ($c = 0$).
3. Найдите (x^*, y^*, μ^*) .
4. Убедитесь, что $\mu^* \geq 0$.

! Место для аналитического решения (ограничение-неравенство) !

Запишите ваши вычисления:

Проверка: $x^{*2}_{\text{безусл.}} + y^{*2}_{\text{безусл.}} = \$ \dots$

≤ 4 ?

Система ККТ: ...

$(x^*, y^*, \mu^*) = \$ \dots$

$J(x^*, y^*) = \$ \dots$

```
In [118... # --- Численная проверка (ограничение-неравенство) ---
...

```

```
In [119... # --- Визуализация с ограничением-неравенством ---
...

```

! Вопрос !

1. Почему множитель ККТ μ должен быть неотрицательным?
Какой физический смысл он несёт?*
2. Что означает условие дополняющей нежёсткости $\mu \cdot c(x, y) = 0$?

Ваш ответ:

Часть 2. Градиентный поиск минимума целевой функции

2.1. Метод Ньютона — Рафсона

Метод Ньютона — Рафсона использует квадратичную аппроксимацию целевой функции для нахождения стационарной точки. Итерационная формула: $\mathbf{p}_{k+1} = \mathbf{p}_k - \mathbf{H}^{-1}(\mathbf{p}_k) \cdot \nabla J(\mathbf{p}_k)$, где $\mathbf{p}_k = (x_k, u_k)^T$ — текущее приближение, $\mathbf{H}$ — матрица Гессе, ∇J — градиент.

Задание:

1. Реализуйте один шаг метода Ньютона — Рафсона, начиная из точки $(x_0, u_0) = (0, 0)$.
2. Убедитесь, что для квадратичной функции сходимость за один шаг. Объясните почему.
3. Сохраняйте все промежуточные точки для визуализации траектории.

! Место для ручного расчёта одного шага метода Ньютона !

Начальная точка: $\mathbf{p}_0 = (0, 0)^T$

$\nabla J(\mathbf{p}_0) = \dots$

$H = \dots$

$H^{-1} = \dots$

$\mathbf{p}_1 = \mathbf{p}_0 - H^{-1} \nabla J(\mathbf{p}_0) = \dots$

```
In [142... def newton_raphson(grad_func, hess_func, p0, tol=1e-10, max_iter=100):
    """
```

Метод Ньютона — Рафсона для минимизации.

Параметры:

grad_func: функция, возвращающая градиент в точке (x, y)
 hess_func: функция, возвращающая матрицу Гессе в точке (x, y)
 p0: начальная точка [x0, u0]
 tol: точность (критерий остановки по норме градиента)
 max_iter: максимальное число итераций

Возвращает:

trajectory: список всех точек [x_k, u_k] на каждой итерации
 """

pass

```
In [121... # --- Запуск метода Ньютона — Рафсона ---
    ...
```

Связь с динамической системой. Градиентный спуск можно интерпретировать как дискретизацию ОДУ $\dot{\mathbf{p}} = -\nabla J(\mathbf{p})$.

Задание на доказательство: Докажите теоретически (используя собственные числа матрицы Гессе), при каких значениях шага γ возникает апериодическая и колебательная сходимость.

Определите критическое значение $\gamma_{\text{кр}}$, выше которого метод расходится.

! Место для ручного расчёта 5 шагов (γ выбрать так, чтобы была аperiodическая сходимость) !

$\mathbf{p}_0 = (0, 0)$, $\nabla J(\mathbf{p}_0) = \dots$

$\mathbf{p}_1 = \dots$

$\nabla J(\mathbf{p}_1) = \dots$

$\mathbf{p}_2 = \dots$

$\mathbf{p}_3 = \dots$

$\mathbf{p}_4 = \dots$

$\mathbf{p}_5 = \dots$

```
In [122... def gradient_descent(grad_func, J_func, p0, gamma, tol=1e-8, max_iter=200):
    """
    Метод наискорейшего спуска.

    Параметры:
        grad_func: функция, возвращающая градиент
        J_func: целевая функция
        p0: начальная точка [x0, u0]
        gamma: шаг (скорость обучения)
        tol: точность
        max_iter: максимальное число итераций

    Возвращает:
        trajectory: список точек
        J_values: список значений J на каждой итерации
    """
    pass
```

```
In [123... # --- Запуск для двух значений gamma ---
p0 = [0.0, 0.0]

gamma_aperiodic = ... # Аperiodическая сходимость
gamma_oscillatory = ... # Колебательная сходимость

traj_ap, J_ap = ..., ...
traj_os, J_os = ..., ...
```

```
In [124... # --- Визуализация траекторий на контурном графике ---
...

```

In [125... # --- Графики сходимости $J(k)$ ---
...]

! Вопрос !

1. Почему метод Ньютона сходится за один шаг, а градиентный спуск за большее? Когда преимущество метода Ньютона теряется?
2. Как визуально отличить аperiodическую сходимость от колебательной на графике траекторий?
3. Что произойдёт, если взять γ большее, чем из найденного диапазона? Объясните, используя собственные числа матрицы Гессе.

Ваш ответ:

2.3. Расходимость и неквадратичные функции

Задание:

Рассмотрите **неквадратичную** функцию Розенброка: $J_R(x, y) = (1 - x)^2 + 100(y - x^2)^2$. Найдите её минимум методами Ньютона и градиентного спуска. Сравните число итераций.

```
In [126... # --- Функция Розенброка ---

def J_rosen(x, y):
    # ===== ВАШ КОД ЗДЕСЬ =====
    return 0

def grad_J_rosen(x, y):
    # ===== ВАШ КОД ЗДЕСЬ =====
    dJdx = 0
    dJdy = 0
    return np.array([dJdx, dJdy])

def hessian_J_rosen(x, y):
    # ===== ВАШ КОД ЗДЕСЬ =====
    H = np.zeros((2, 2))
    return H

# Запустите оба метода для функции Розенброка из точки (-1, 1)
# Сравните число итераций
# ===== ВАШ КОД ЗДЕСЬ =====
```


! Вопрос !

Почему для функции Розенброка метод градиентного спуска сходится крайне медленно? Как связано число обусловленности матрицы Гессе со скоростью сходимости?

Ваш ответ:

Часть 3. Численный поиск экстремалей вариационных задач

3.1. Связь между вариационным исчислением и статической оптимизацией

В курсе мы рассматривали задачи вариационного исчисления: нахождение функции $y(x)$, доставляющей экстремум функционалу $\mathcal{J}[y] = \int_a^b L(x, y, y') dx$, $y(a) = A$, $y(b) = B$.

Экстремаль определяется **уравнением Эйлера — Лагранжа**: $\frac{\partial L}{\partial y} - \frac{d}{dx} \frac{\partial L}{\partial y'} = 0$.

Достаточное условие минимума обеспечивается выполнением **условия Лежандра** ($L_{y'y'} > 0$) и отсутствием сопряжённых точек (**уравнение Якоби**).

Идея: дискретизация

Разобьём отрезок $[a, b]$ на N равных частей: $x_i = a + ih$, $h = \frac{b-a}{N}$, $i = 0, 1, \dots, N$.

Заменяем непрерывную функцию $y(x)$ набором значений $y_0, y_1, \dots, y_N$ в узлах сетки, а производную — конечной разностью: $y'(x_i) \approx \frac{y_{i+1} - y_i}{h}$.

Тогда функционал превращается в **функцию конечного числа переменных**: $\mathcal{J}[y] \approx \Phi(y_1, y_2, \dots, y_{N-1}) = \sum_{i=0}^{N-1} L(x_i, y_i, \frac{y_{i+1} - y_i}{h}) h$, где $y_0 = A$ и $y_N = B$ фиксированы граничными условиями, а свободными переменными являются $y_1, y_2, \dots, y_{N-1}$.

Таким образом, задача вариационного исчисления **сводится** к задаче статической оптимизации, для которой можно применить **градиентный спуск** или **метод Ньютона**!

3.2. Поиск экстремали для функционала $\int_0^1 (p y'^2 + q y^2) dx$

Рассмотрим задачу: $\mathcal{J}[y] = \int_0^1 (y'^2 + y^2) dx \rightarrow \min$,
 $\text{quad } y(0) = 0, \text{quad } y(1) = 1$.

! Место для аналитического вывода !

Самостоятельно выведите уравнение Эйлера — Лагранжа и найдите решение. Проверьте достаточное условие.

Ваш вывод:

3.3. Дискретизация и численное решение

Реализуем поиск экстремали численно. Подставив $L = p y'^2 + q y^2$ и $y' \approx \frac{y_{i+1} - y_i}{h}$, получим дискретный функционал: $\Phi(y_1, \dots, y_{N-1}) = \sum_{i=0}^{N-1} \left[\left(\frac{y_{i+1} - y_i}{h} \right)^2 + y_i^2 \right] h$.

Градиент по y_j ($j = 1, \dots, N-1$): $\frac{\partial \Phi}{\partial y_j} = \frac{2(y_j - y_{j-1})}{h} - \frac{2(y_{j+1} - y_j)}{h} + 2 y_j h = \frac{2}{h} (-y_{j-1} + 2y_j - y_{j+1}) + 2 y_j h$.

Задание:

1. Реализуйте вычисление Φ и его градиента.
2. Найдите экстремаль методом градиентного спуска.
3. Найдите экстремаль методом Ньютона.
4. Сравните численное решение с аналитическим.

```
In [127... # --- Параметры дискретизации ---
N = 20      # число интервалов
a, b = 0.0, 1.0
A, B = 0.0, 1.0 # граничные условия y(0) = 0, y(1) = 1
h = (b - a) / N
x_grid = np.linspace(a, b, N + 1) # узлы сетки

# --- Аналитическое решение ---
y_exact = ...

print(f"Число узлов: {N + 1}, шаг h = {h:.4f}")
print(f"Число свободных переменных: {N - 1}")
```

Число узлов: 21, шаг h = 0.0500

Число свободных переменных: 19

```
In [128... def Phi(y_inner):
    """
    Дискретный функционал  $\Phi(y_1, \dots, y_{N-1})$ .
    y_inner: массив из N-1 значений (внутренние узлы).
    """
    pass

def grad_Phi(y_inner):
    pass
```

```
In [129... # --- Проверка градиента численным дифференцированием ---
... 
```

```
In [130... # --- Градиентный спуск ---
... 
```

```
In [131... # --- Метод Ньютона ---
... 
```

```
In [132... # --- Сравнение с аналитическим решением, график погрешностей ---
... 
```

```
In [133... # --- График сходимости ---
... 
```

! Вопрос !

1. Сколько итераций потребовалось методу Ньютона для нелинейной задачи? Сравните с линейной.
2. Проверьте условие Лежандра для данного функционала. Гарантирует ли оно минимум?
3. Как соотносятся подходы: решение уравнения Эйлера — Лагранжа (дифференциальное уравнение) и минимизация дискретного функционала (статическая оптимизация)? Когда каждый из них предпочтителен?

Ваш ответ:

3.5. Исследование влияния числа узлов

Проведите эксперимент: для линейной задачи ($\mathcal{J}[y] = \int_0^1 (y'^2 + y^2) dx$) постройте зависимость максимальной погрешности от числа узлов N .

```
In [134... ...
```

! Вопрос !

1. Как связана погрешность дискретизации (отличие численного решения от аналитического) с числом узлов N ? Проведите эксперимент для $N = 5, 10, 20, 50, 100$. Какой порядок сходимости по h вы наблюдаете? Объясните.
2. Почему для линейного уравнения Эйлера — Лагранжа метод Ньютона сходится за один шаг? Будет ли это так для нелинейного уравнения?
3. Как выбор шага γ зависит от N ? Что происходит при фиксированном γ и росте N ?

Ваш ответ:

Часть 4. Оптимизация траектории робота на поверхности

4.1. Постановка задачи

Рассмотрим задачу планирования движения робота по неровной поверхности. Пусть поверхность задана функцией высот $z = h(x, y)$, а траектория робота в проекции на плоскость XY описывается кривой $y = y(x)$, соединяющей точки $A(a, c)$ и $B(b, d)$.

Цель — найти такую траекторию, которая минимизирует интегральный функционал, отражающий «стоимость» движения:
$$\mathcal{J}[y] = \int_a^b \left(A + B \|\nabla h(x, y(x))\| \right) \sqrt{1 + (y'(x))^2} \, dx \rightarrow \min,$$
 где:

- $A > 0$ — базовая стоимость перемещения (трение, энергопотребление на ровной поверхности);
- $B \geq 0$ — коэффициент «чувствительности» к рельефу;
- $\|\nabla h\| = \sqrt{\left(\frac{\partial h}{\partial x}\right)^2 + \left(\frac{\partial h}{\partial y}\right)^2}$ — модуль градиента поверхности (крутизна склона);
- $\sqrt{1 + (y')^2} \, dx$ — элемент длины дуги траектории.

Данный функционал является обобщением задачи о брахистохроне и геодезических линиях. При $B = 0$ задача сводится к поиску кратчайшего пути. При $B > 0$ робот «предпочитает» пологие участки, даже если путь удлиняется.

Дискретизация функционала

Разобьём отрезок $[a, b]$ на N равных частей: $x_i = a + i \cdot \Delta x$, $\Delta x = \frac{b-a}{N}$, $i = 0, \dots, N$. Заменим непрерывную функцию $y(x)$ набором значений $y_0, y_1, \dots, y_N$, где $y_0 = c$, $y_N = d$ фиксированы граничными условиями.

Производную аппроксимируем конечной разностью: $y'(x_i) \approx \frac{y_{i+1} - y_i}{\Delta x}$. Тогда дискретный аналог функционала принимает вид:
$$\Phi(y_1, \dots, y_{N-1}) = \sum_{i=0}^{N-1} \left[A + B \cdot |\nabla h(x_i, y_i)| \right] \cdot \sqrt{1 + \left(\frac{y_{i+1} - y_i}{\Delta x} \right)^2} \cdot \Delta x.$$

Таким образом, задача вариационного исчисления сводится к задаче **статической оптимизации** функции Φ от $N-1$ переменных, которую можно решать методами градиентного спуска или Ньютона, изученными ранее.

! Место для аналитической подготовки !

1. Выведите выражение для градиента дискретного функционала $\frac{\partial \Phi}{\partial y_j}$, $j = 1, \dots, N-1$.
2. Объясните, почему для вычисления $|\nabla h|$ необходимо знать частные производные $\frac{\partial h}{\partial x}$ и $\frac{\partial h}{\partial y}$.
3. Как изменится функционал, если добавить штраф за отклонение от плоскости $z = 0$?

Ваш вывод:

4.2. Генерация поверхности с помощью шума Перлина

```
In [135... def generate_perlin_surface(x_grid, y_grid, params):
    freq = params.get('FREQ', 1.0)
    h = np.sin(freq * x_grid) * np.cos(freq * y_grid) + 0.5 * np.sin(2 * freq * x_grid)
    dh_dx = freq * np.cos(freq * x_grid) * np.cos(freq * y_grid) + freq * np.sin(2 * freq * x_grid)
    dh_dy = -freq * np.sin(freq * x_grid) * np.sin(freq * y_grid) - freq * np.sin(2 * freq * x_grid)
    return h, dh_dx, dh_dy
```

```
In [136... # --- Параметры области и сетки ---
x_min, x_max = -5.0, 5.0
y_min, y_max = -5.0, 5.0
nx, ny = 100, 100

x_surf = np.linspace(x_min, x_max, nx)
y_surf = np.linspace(y_min, y_max, ny)
```

```

X_surf, Y_surf = np.meshgrid(x_surf, y_surf)

# --- Параметры варианта ---
params_variant = params

# --- Генерация поверхности ---
H_surf, dHdx_surf, dHdy_surf = generate_perlin_surface(X_surf, Y_surf, param

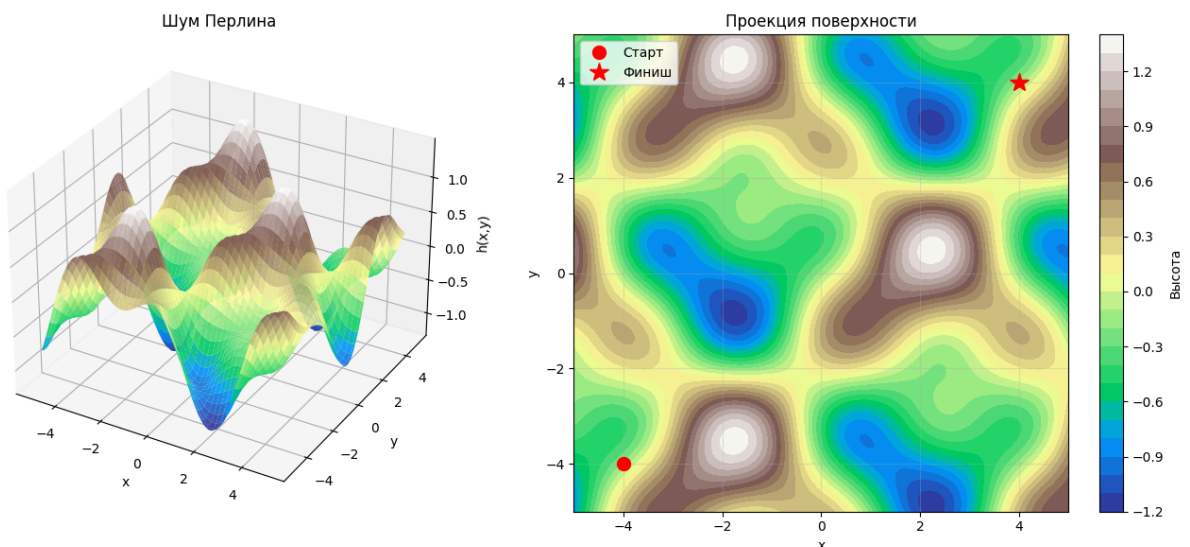
# --- Визуализация поверхности ---
fig = plt.figure(figsize=(14, 6))

# 3D-график
ax1 = fig.add_subplot(121, projection='3d')
surf = ax1.plot_surface(X_surf, Y_surf, H_surf, cmap='terrain', alpha=0.9, l
ax1.scatter([params_variant['x_start'], params_variant['x_end']],
            [params_variant['y_start'], params_variant['y_end']],
            [generate_perlin_surface(params_variant['x_start'], params_varia
            color='red', s=100, zorder=5)
ax1.set_xlabel('x')
ax1.set_ylabel('y')
ax1.set_zlabel('h(x,y)')
ax1.set_title('Шум Перлина')

# Контурный график
ax2 = fig.add_subplot(122)
contour = ax2.contourf(X_surf, Y_surf, H_surf, levels=30, cmap='terrain')
plt.colorbar(contour, ax=ax2, label='Высота')
ax2.plot(params_variant['x_start'], params_variant['y_start'], 'ro', markers
ax2.plot(params_variant['x_end'], params_variant['y_end'], 'r*', markersize=
ax2.set_xlabel('x')
ax2.set_ylabel('y')
ax2.set_title('Проекция поверхности')
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```



4.3. Дискретизация траектории и вычисление функционала

Задание:

1. Создайте массив узлов траектории `x_traj` на отрезке `[x_start, x_end]` с `N_nodes` точками.
2. Реализуйте функцию `compute_functional(y_inner, x_traj, h_func, grad_h_func, params)`, вычисляющую значение дискретного функционала Φ .
3. Реализуйте функцию `compute_gradient(y_inner, x_traj, h_func, grad_h_func, params)`, вычисляющую градиент $\nabla \Phi$ по внутренним узлам $y_1, \dots, y_{N-1}$.

Важно: Функции `h_func` и `grad_h_func` должны принимать скалярные координаты `(x, y)` и возвращать значение высоты и градиент в этой точке. Для интерполяции значений с сетки `H_surf` используйте `scipy.interpolate.RegularGridInterpolator` или `np.interp`.

```
In [137... def h_interp(x, y):
    pass

def grad_h_interp(x, y):
    pass
```

```
In [138... def compute_functional(y_inner, x_traj, h_func, grad_h_func, params):
    """
    Вычисление дискретного функционала  $\Phi$ .

    Параметры:
        y_inner: массив внутренних узлов траектории  $[y_1, \dots, y_{N-1}]$ 
        x_traj: массив всех узлов по x (включая граничные)
        h_func, grad_h_func: функции для вычисления высоты и градиента
        params: словарь параметров (A_weight, B_weight, ...)

    Возвращает:
        phi: значение функционала
    """
    pass

def compute_gradient(y_inner, x_traj, h_func, grad_h_func, params):
    """
    Вычисление градиента функционала по внутренним узлам.

    Возвращает:
        grad: массив длины N-1 с частными производными  $d\Phi/dy_j$ 
    """
```

.....

pass

4.4. Численная оптимизация траектории

Задание:

1. Используя метод градиентного спуска, найдите оптимальную траекторию, минимизирующую функционал Φ .
2. Сохраняйте значения функционала и траектории на каждой итерации для последующей визуализации процесса сходимости.
3. Подберите шаг обучения γ экспериментально.

```
In [139... # --- Подготовка данных для оптимизации ---
x_traj = np.linspace(params_variant['x_start'], params_variant['x_end'], par
y_init = np.linspace(params_variant['y_start'], params_variant['y_end'], par

gamma_opt = 0.01 # шаг градиентного спуска (подберите!)
max_iter_opt = 2000
tol_opt = 1e-8

# --- Градиентный спуск ---
...
```

Out[139... Ellipsis

4.5. Визуализация и интерпретация результатов

Задание:

1. Постройте финальную визуализацию: 3D-график поверхности с нанесённой оптимальной траекторией и 2D-контурный график с проекцией пути.
2. Создайте анимацию или серию кадров, показывающих эволюцию траектории в процессе оптимизации (используйте `trajectory_history`).
3. Постройте график сходимости: значение функционала Φ от номера итерации.
4. Ответьте на вопросы для интерпретации.

In [140... ...

Out[140... Ellipsis

! Вопросы для интерпретации результатов !

1. Почему оптимальный путь имеет именно такую форму? Как на него повлияли коэффициенты A и B ? Что произошло бы, если

- бы B было значительно больше?
2. Сколько итераций потребовалось для сходимости? Как выбор шага γ влияет на скорость и устойчивость процесса? Почему использование аналитического градиента предпочтительнее численного?

Ваши ответы:

4.6. Модификация функционала

1. Модифицируйте функционал, добавив штраф за резкие изменения направления: $\mathcal{J}_{\text{new}} = \mathcal{J} + C \int (y''(x))^2 dx$. Реализуйте дискретизацию второго производного и соответствующий градиент.
2. Исследуйте, как параметр C влияет на «плавность» оптимальной траектории.
3. Попробуйте использовать метод Ньютона для минимизации функционала. Для этого реализуйте вычисление (или аппроксимацию) матрицы Гессе дискретного функционала.
4. Сравните время работы и точность методов градиентного спуска и Ньютона для вашей задачи.

In [141... ...

Out[141... Ellipsis